İşletim Sistemleri Hafta 4 Lab Çalışması
20.10.2026

Tüm sorular önce kod sonra kod çıktısı olacak şekilde cevaplanmıştır.

1) Her öğrencinin toplam skorunu (2. ve 3. kolonun toplamı) gösterin.

```
  GNU nano 9.0
dosya="veri.txt"

awk -F ';' '{print $1, $2+$3}' $dosya
```

```
┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ ./program1.sh
Serkan 30
Merve 20
Emre 10
Derya 47
Onur 25
Esra 31
Esra 23
Can 22
Mehmet 34
Tuba 25
Zeynep 35
Merve 17
Cem 12
Tuba 26
Gül 23
Seda 28
Cem 16
Zeynep 14
Kemal 16
Ali 39
Ahmet 17
Gül 34
Deniz 14
Esra 8
Seda 32
Yasemin 27
Veli 21
Ali 24
```

2) İkinci kolonunda 20 ve üzerinde değer olan öğrencilerin isimlerini ve bu değerleri listeleyin.

```
  GNU nano 9.0
dosya="veri.txt"

awk -F ';' '$2 ≥ 20 {print $1, $2}' $dosya
```

```
┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ ./program1.sh
Mehmet 20
```

3) Üçüncü kolondaki değeri 11 olan öğrencilerin isimlerini listeleyin.

```
  GNU nano 9.0
dosya="veri.txt"

awk -F ';' '$3 == 11 {print $1}' $dosya
```

```
┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ ./program1.sh
Merve
Can
Tuba
Zeynep
```

4) İkinci kolondaki değeri 10 ile 15 arasında olan öğrencilerin isimlerini listeleyin.

```
  GNU nano 9.0
dosya="veri.txt"

awk -F ';' '$2 >= 10 && $2 <= 15{print $1}' $dosya
```

```
┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ ./program1.sh
Serkan
Onur
Esra
Can
Tuba
Cem
Tuba
Gül
Veli
```

5) İsmi çift kez geçen öğrencilerin toplamda kaç kere geçtiğini bulun.

```
  GNU nano 9.0                                  program1.sh
dosya="veri.txt"

cut -d ';' -f1 $dosya | sort | uniq -c | awk '$1 == 2 {toplam += $1} END {print toplam+0}'
```

```
┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ ./program1.sh
14
```

(Dolayısıyla 7 öğrencinin ismi 2 defa vardır ve totalde 14 kere geçmiştir.)

6) Her öğrencinin adı, ikinci kolon ve üçüncü kolonun toplamını öğrenci_adi: toplam formatında yazdırın.

```
  GNU nano 9.0
dosya="veri.txt"

awk -F ';' '{print $1 ":" $2+$3}' $dosya
```

```
└─$ ./program1.sh
Serkan:30
Merve:20
Emre:10
Derya:47
Onur:25
Esra:31
Esra:23
Can:22
Mehmet:34
Tuba:25
Zeynep:35
Merve:17
Cem:12
Tuba:26
Gül:23
Seda:28
Cem:16
Zeynep:14
Kemal:16
Ali:39
Ahmet:17
Gül:34
Deniz:14
Esra:8
Seda:32
Yasemin:27
Veli:21
Ali:24
```

7) İkinci kolondaki en küçük değere sahip öğrencinin ismini yazdırın.

```
  GNU nano 9.0                                    program1.sh
dosya="veri.txt"

awk -F ';' 'NR == 1 || $2 < min {min=$2; isim=$1} END {print isim}' $dosya
```

```
┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ ./program1.sh
Yasemin
```

8) Tüm öğrencilerin ikinci ve üçüncü kolonlarının toplamını hesaplayıp, bu toplamı 20 den büyük olanları yazdırın.

```
  GNU nano 9.0
dosya="veri.txt"

awk -F ';' '$2+$3 > 20 {print $1, $2+$3}' $dosya
```

```
┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ ./program1.sh
Serkan 30
Derya 47
Onur 25
Esra 31
Esra 23
Can 22
Mehmet 34
Tuba 25
Zeynep 35
Tuba 26
Gül 23
Seda 28
Ali 39
Gül 34
Seda 32
Yasemin 27
Veli 21
Ali 24
```

9) İsmi "A" harfi ile başlayan, ikinci sütunu 3'ten büyük ve üçüncü sütunu 10'dan küçük olan satırları filtrelenmis.txt isimli bir dosyaya yazdırın ve kaç adet olduklarını ekrana yazdırın.

```
  GNU nano 9.0                                    program1.sh
dosya="veri.txt"

awk -F ';' '$1 ~ /^A/ && $2 > 3 && $3 < 10' $dosya > filtrelenmis.txt
wc -l < filtrelenmis.txt
```

```
┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ nano program1.sh

┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ ./program1.sh
1

┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ ls
filtrelenmis.txt  program1.sh  veri.txt

┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ cat filtrelenmis.txt
Ahmet;17;0
```

10) İsmi "M" harfi ile başlayan ve toplam skoru 25'ten küçük olan öğrencilerin bilgilerini m_ogrenciler.txt dosyasına yazdırın. Toplamda kaç öğrenci olduğunu ekrana yazdırın.

```
  GNU nano 9.0                                          program1.sh
dosya="veri.txt"

awk -F ';' '$1 ~ /^M/ && $2+$3 < 25' $dosya > m_ogrenciler.txt
wc -l < m_ogrenciler.txt
```

```
┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ ./program1.sh
2

┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ ls
filtrelenmis.txt  m_ogrenciler.txt  program1.sh  veri.txt

┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ cat m_ogrenciler.txt
Merve;9;11
Merve;5;12
```

11) İsmi "S" harfi ile başlayan ve üçüncü sütunu 20'den büyük olan satırların isimlerini isimler.txt dosyasına yazdırın ve kaç isim olduğunu ekrana yazdırın.

```
  GNU nano 9.0                                               prog
dosya="veri.txt"

awk -F ';' '$1 ~ /^S/ && $3 > 20' $dosya > isimler.txt
wc -l < isimler.txt
```

```
┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ ./program1.sh
2

┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ ls
filtrelenmis.txt  isimler.txt  m_ogrenciler.txt  program1.sh  veri.txt

┌──(kali㉿kali)-[~/isletimSistemleriLab/Lab_Hafta4_Calisma]
└─$ cat isimler.txt
Seda;2;26
Seda;6;26
```

1. Chrome Browser yapısını araştırınız. Web tarayıcısının tek bir process (işlem) olarak çalışmasının ve birden fazla process (işlem) olarak çalışmasını avantajları ve dezavantajları açısından karşılaştırınız.

Cevap: Tarayıcı, birden fazla process olarak çalıştığında başlatılan her işlem için bellekte bir yer ayrılır, birbirinden bağımsız olur ve birbirlerini etkilemezler. Böylece sayfaların yüklenmesi, görüntülenmesi, kullanıcının sayfayla etkileşimi vs. daha hızlı olur. Temel olarak bir performans artışı sağlar ve herhangi bir sekmenin arızalanmasına karşın diğer sekmelerin/işlemlerin etkilenmesini önler.

2. Non-multi-programming, multi-programming, multi-processing, multi-tasking-multi-threading yapılarını açıklayınız.

Cevap:

- **Non-multi-programming:** Bir anda tek bir program çalışır/bellekte tutulur, diğerleri o programın bitmesini bekler. CPU bazen I/O beklerken boşta kalabilir.
- **Multi-programming:** Bir anda birden fazla program birlikte çalışır/bellekte tutulur. CPU'nun boş durma süresi daha azdır, I/O beklerken CPU diğer programları çalıştırır.
- **Multi-processing:** İşlemci çekirdeklerinin veya işlem birimlerinin process'leri aynı anda yürütebilmesidir.
- **Multi-tasking:** Birden fazla görev kısa aralıklarla CPU üzerinde çalışır, uygun oldukça swapping gerçekleşir.
- **Multi-threading:** Tek bir process içinde birden fazla iş parçacığı yani thread'in çalıştırılmasıdır. Aynı process'in threadleri genellikle adres alanını ve kaynakların bir kısmını paylaşırlar.

3. Virtualization (Sanallaştırma) kavramını donanım kaynakları ve işletim sistemi açısından nasıl farklılıklar oluştuğunu belirterek açıklayınız.

Cevap: Farklı sanal makineler aynı cihazda kullanıldığında CPU, RAM ve disk alanı gibi donanımları, kendilerine tahsis edildiği ölçüde, paylaşırlar. Her sanal makineye farklı işletim sistemleri verilebilir, genel olarak 2 sanal makinenin işletim sistemleri birbirlerinden bağımsız/izole olarak çalışırlar. Mesela 1. sanal makine windows kullanırken diğeri Linux kullanabilir.